

Desarrollo de Aplicaciones para Ciencia de Datos

Proyecto de la asignatura. Sprint 2:
Incorporación de datos a la arquitectura del sistema.

Duración: 3 semanas

Modalidad: Trabajo en equipo

1. Objetivo

Se propone este sprint la implementación del patrón **Publisher/Subscriber** partiendo del trabajo realizado en el **Sprint 1**.

La idea es que los módulos *feeders* **publiquen eventos** en un *broker* (ActiveMQ) y un módulo *Event Store Builder* los **consume y almacene** de forma ordenada y persistente.

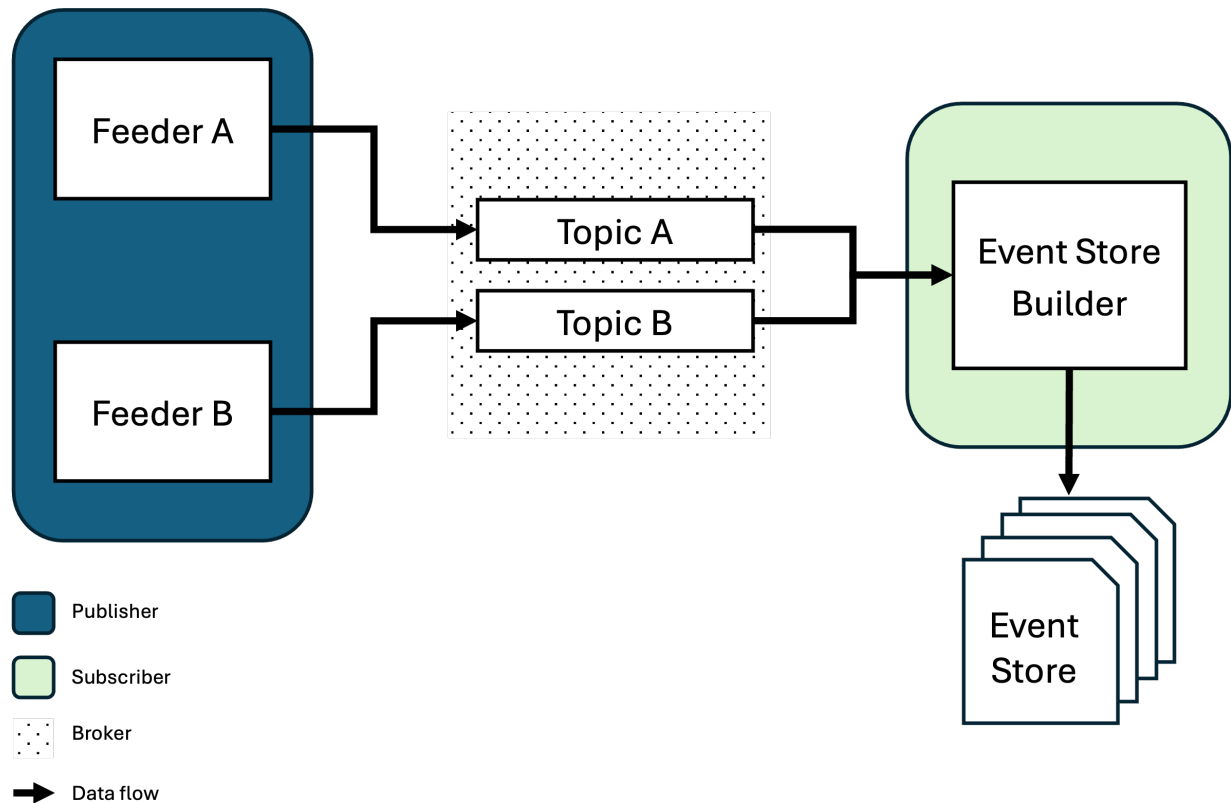


Figura 1: Arquitectura Publisher/Subscriber propuesta en el Sprint 2.

Cada elemento (*feeders* y *Event Store Builder*) será un módulo del proyecto. El *broker* se instalará como una aplicación/servicio en el sistema operativo (no es un módulo Java del proyecto).

2. Requisitos del Proyecto

2.1. Requisitos operacionales

1. El programa podrá funcionar en local y mediante una interfaz de usuario en modo texto (si fuera necesario informar de algo).
2. Estará programado en Java, usando la versión 21.
3. El proyecto debe estar desarrollado en el entorno de programación IntelliJ.

2.2. Requisitos funcionales

Los requisitos funcionales se organizan por componente:

1. **Feeders (publishers)**. Cada módulo *feeder* (uno por fuente externa seleccionada en el Sprint 1) obtiene datos con la frecuencia especificada en el **Sprint 1** y publica un evento en formato JSON tras cada captura. En el caso de una fuente meteorológica, se tratará de predicciones para las localizaciones escogidas.

El evento tendrá la siguiente estructura mínima:

- **ts**: timestamp en UTC asociado al dato (p. ej., instante de captura o de la predicción).
- **ss**: identificador de la fuente/módulo que produce el dato (p. ej., **feeder A**).
- Resto de atributos del evento (*payload*) según la fuente.

Los *topics* deben tener el nombre del tipo de evento que publican (p. ej., **Weather**, **HotelPrice**, etc.). Se puede añadir un *namespace* si fuera necesario.

2. **Broker**. Se usará **ActiveMQ** como *broker* de mensajería (existen otras alternativas como Kafka, RabbitMQ, Mosquitto, etc., pero no se usarán en este sprint).
3. **Event Store Builder (subscriber)**. Este módulo se suscribe de forma **durable** a los *topics* publicados por los *feeders* y va almacenando los eventos consumidos en un *event store* organizado temporalmente. La estructura de directorios a seguir es:

`eventstore/{topic}/{ss}/{YYYYMMDD}.events`

Donde **topic** es el *topic* de origen del mensaje y **YYYYMMDD** es el año-mes-día obtenido a partir del **ts** del evento. La extensión **.events** identifica un fichero con eventos de un

día concreto (por ejemplo, `20231103.events`). Los eventos se irán añadiendo (*append*) al final del fichero, con **un evento por línea** (formato JSON Lines/NDJSON).

La suscripción debe ser **durable** para que, si el *Event Store Builder* se detiene temporalmente, al reiniciar pueda recuperar los mensajes que no se hayan consumido.

3. Cronograma

3.1. Semana 1: configuración e integración del broker

Objetivo: Instalar y configurar ActiveMQ, y modificar los módulos *feeders* para enviar eventos al *broker*.

Actividades:

1. Instalar ActiveMQ en el equipo de desarrollo siguiendo la documentación oficial.
2. Verificar que ActiveMQ está corriendo correctamente en el puerto 61616.
3. Actualizar los módulos *feeders* para incluir la dependencia de ActiveMQ (`org.apache.activemq:activemq-client:5.15.12`).
4. Diseñar la estructura del evento en una clase Java.
5. Transformar los datos que vienen de la fuente externa (API y/o scraping) en instancias de la clase elegida como evento.
6. Implementar la lógica de serialización a JSON usando Gson o Jackson.
7. Enviar eventos al *topic* que corresponda tras cada captura.

3.2. Semana 2: desarrollo del event store builder

Objetivo: Implementar el consumidor que se suscribe al *broker* y almacena los eventos de manera organizada.

Actividades:

1. Crear el módulo *Event Store Builder* y agregar las dependencias necesarias.
2. Implementar la suscripción duradera a los diferentes *topics*.

3. Recibir eventos desde ActiveMQ y deserializarlos (por ejemplo, a objetos `JsonObject` o a una clase de evento).
4. Estructurar el almacenamiento de eventos siguiendo el formato:

`eventstore/{topic}/{ss}/{YYYYMMDD}.events`

5. Abrir y escribir en archivos de eventos asegurando que cada línea sea un evento en formato JSON.
6. Verificar la correcta organización de los eventos en la estructura de directorios.

3.3. Semana 3: pruebas, optimizaciones y documentación

Objetivo: Asegurar el correcto funcionamiento de los módulos y documentar el proyecto.

Actividades:

1. Probar la integración entre los *feeders* y el *broker* ActiveMQ asegurando que los eventos se envían correctamente.
2. Probar la suscripción del *Event Store Builder* verificando que los eventos se almacenan en los archivos correspondientes.
3. Manejar errores y reconexiones en caso de fallos en la comunicación con el *broker*.
4. Refactorizar el código para mejorar su modularidad y eficiencia, asegurando que cumple estándares de desarrollo y diseño vistos en clase (*clean code*).

4. Recursos

1. Documentación oficial de ActiveMQ:
 - Instalación y configuración: guía de inicio
 - Descarga del binario: descarga
 - Uso de ActiveMQ con Java: ejemplo “Hello World” (JMS)
2. Librerías necesarias:
 - ActiveMQ Client: `org.apache.activemq:activemq-client:5.15.12`

- Gson para serialización JSON:
`com.google.code.gson:gson:2.8.9`
- Jackson para serialización JSON (alternativa a Gson):
`com.fasterxml.jackson.core:jackson-databind`

3. Herramientas recomendadas:

- Postman/Talend Api Tester: Para probar las solicitudes a los servicios remotos.
- Consola de ActiveMQ: Para visualizar los mensajes en ActiveMQ.

5. Consejos para el desarrollo

- **Dividan el trabajo entre los miembros de la pareja:** uno puede enfocarse en los *feeders* y el otro en *Event Store Builder*.
- **Prueben ActiveMQ con ejemplos básicos** antes de integrarlo al código del proyecto.
- **Manejen correctamente las excepciones** en la conexión con ActiveMQ para evitar bloqueos.
- **Utilicen logs para depurar:** agreguen `System.out.println()` o una librería como SLF4J para visualizar eventos enviados y recibidos.
- **Verifiquen la correcta serialización/deserialización** de eventos con Gson/Jackson antes de enviarlos al *broker*.
- **Prueben la estructura de almacenamiento** generando eventos de prueba y asegurándose de que se guardan correctamente en los archivos correspondientes.
- **Hagan commits regularmente** para evitar pérdida de avances.
- **Lleven sus dudas a las sesiones prácticas:** no esperen al último momento para preguntar sobre errores o problemas.